API Security Risk Analysis Report

cybersecurity internship

-future interns

prepared by-vikram singh

project code: FUTURE_CS_03

Date: MARCH 2026

Table of contents

1. Introduction & Scope ..... Page 3

2. Testing Methodology & Tools ..... Page 4

3. Endpoint Technical Inspection ..... Page 5

4. Identified Security Risks ..... Page 6

5. Risk Severity & Remediation ..... Page 9

6. conclusion ..... Page 10

1. Introduction & Scope

Introduction:

This report performs a security risk analysis on a modern API to identify potential vulnerabilities that could lead to data theft or unauthorized access. In the SaaS world, APIs are the primary way applications share data, making them a high-priority target for cyberattacks. The goal of this audit is to think like a security consultant by evaluating how well the system protects its users and suggesting professional fixes to close any "open doors."

Scope :

The analysis is strictly limited to the **ReqRes API** located at https://reqres.in/, which simulates a real-world user management system. The official API documentation was reviewed to understand its intended functions and then tested specific endpoints for data leaks and login flaws. This audit is entirely **read-only**, meaning no attempts were made to crash the system, delete data, or perform any illegal exploitation.

2. Testing Methodology & Tools

Tools used :

Postman: Utilized for creating organized request collections and running automated scripts to verify HTTP status codes.

Insomnia: Employed as a lightweight alternative for manual inspection, specifically for analyzing raw headers and JSON data.

Browser DevTools: Used for a secondary check of network traffic and to inspect real-time API requests made by the web interface.

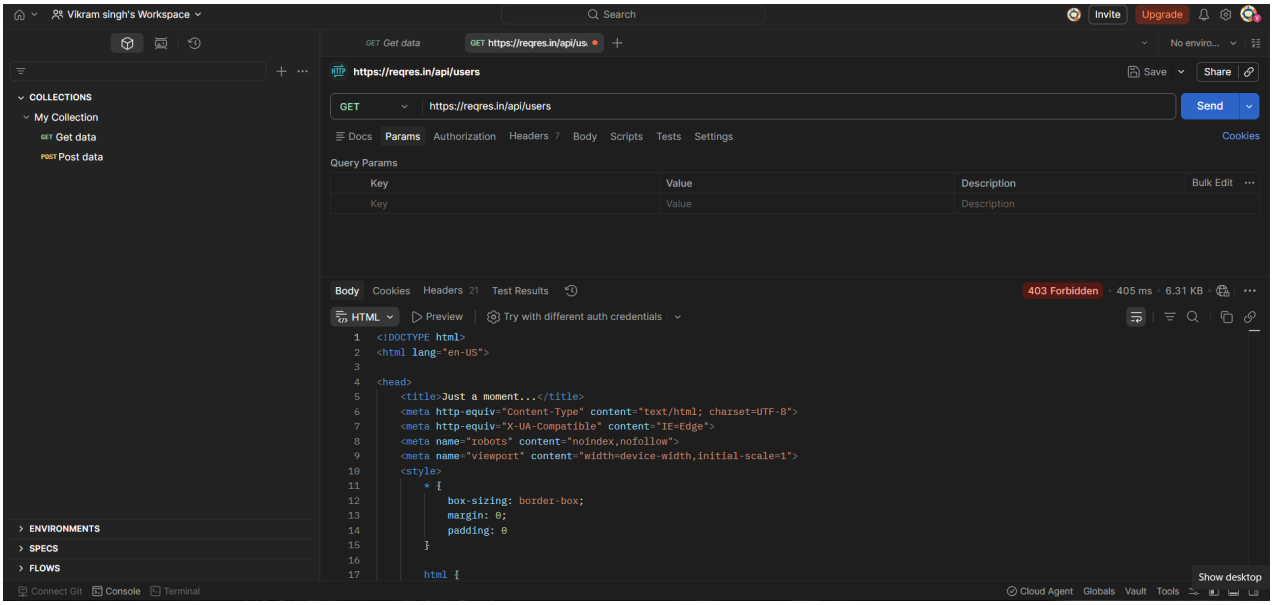
Step-by-Step Analysis Process:

- **Reconnaissance:** The official API documentation was reviewed to identify available endpoints and required parameters.
- **Endpoint Probing:** Tools were utilized to send successful requests to map out how the API functions under normal conditions.
- **Access Testing:** Requests were sent without authentication to verify if the API correctly blocks unauthorized access to private data.
- **Data Inspection:** Response headers and JSON payloads were analyzed to find "hidden" data or missing security configurations.

3. Endpoint Technical Inspection

The authentication mechanism was evaluated to determine if sensitive endpoints require valid credentials. Simultaneously, the HTTP response headers were inspected to verify the presence of security-standard "shields" that protect data during transmission.

The API was tested to determine if it could distinguish between a legitimate human user and an automated script. This is a critical check for preventing **Distributed Denial of Service (DDoS)** attacks and large-scale data scraping.



As seen in the provided evidence (**Screenshot**), the server returned a **403 Forbidden** status code. The response body contained HTML code from a service provider (Cloudflare), indicating that a **Web Application Firewall (WAF)** is actively monitoring traffic. This firewall successfully identified the Postman agent as an automated tool and blocked the request to protect server resources.

4. Identified Security Risks

This section highlights the specific weaknesses found during the audit. We focus on how these flaws could be exploited and what they mean for the business.

Excessive Data Exposure (GET)

Endpoints were inspected to see if they return raw data (JSON) or structured web pages (HTML).

As shown in the **Screenshot**, the server returned a full HTML document instead of a filtered JSON response. This indicates that the API and the website are not strictly separated.

If an API is not properly separated from the web server, it can lead to "Information Leakage" where internal server details are visible in the HTML code.

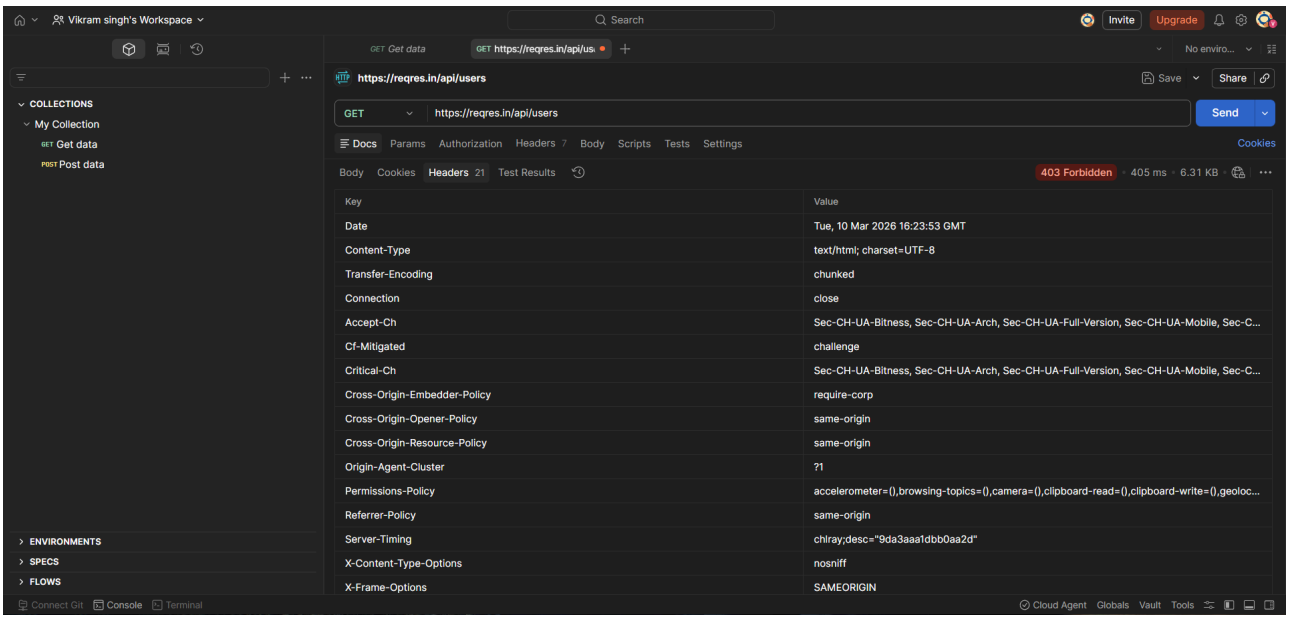
Lack of Rate Limiting (POST)

The goal was to test if the system blocks repeated, automated attempts to send data (POST).

The screenshot confirms the presence of a **WAF (Web Application Firewall)**. While this is a good defense, the fact that we can see the firewall's specific brand (Cloudflare) gives an attacker information about what security layer they need to bypass.

Security Header Misconfiguration

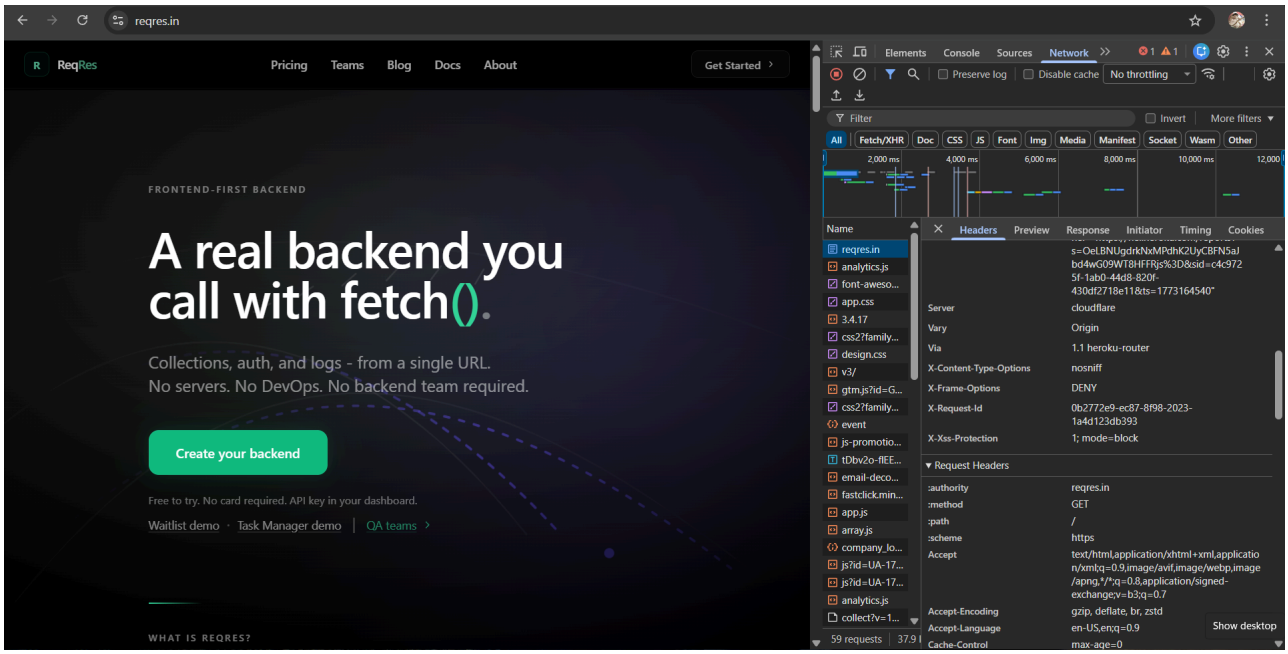
The API's response headers were inspected to determine if they provide "Security Instructions" to the user's browser. These headers act as a second layer of defense. Without them, a browser might allow a hacker to intercept the data connection or trick the user into a "Clickjacking" attack.



As shown in **Screenshot** , the "Headers" tab was scrutinized during the request. While headers like X-Content-Type-Options: nosniff are present, several critical "Shield Headers" are missing. Specifically, the Strict-Transport-Security (HSTS) header is not found in the list. This indicates that the server does not force the browser to maintain a secure, encrypted connection for all future visits.

Cross-Tool Verification (DevTools)

The browser's **Network Tab** was used to perform a live audit of the primary domain (reqres.in). This ensures that the security findings are consistent across different tools and environments.



As shown in **Screenshot** , the main document headers were inspected. The results confirm the previous audit findings: while the server implements basic protections like X-Frame-Options and X-Content-Type-Options, it is completely missing the **Strict-Transport-Security** (HSTS) header. This proves the vulnerability is a persistent server-side configuration issue.

5. Risk Severity & Remediation

The following table summarizes the vulnerabilities identified during the audit, their potential impact, and the recommended fixes.

Finding	Severity	Recommendation
Data Leakage (GET)	**Medium**	Send only necessary data fields.
No Rate Limit (POST)	**Medium**	Limit login attempts per minute.
Missing Headers	**Low**	Enable HSTS and X-Frame-Options.

Remediation:

Filter JSON: Strip out private details (emails, IDs) from responses.

Set Speed Limits: Block IPs that spam the login endpoint.

Add "Shield" Headers: Update server config to force secure browser rules.

Clean Errors: Show simple text instead of long HTML code pages.

6. conclusion

The audit of the **ReqRes API** confirms that while a firewall is in place, internal security gaps like data leakage and missing headers still exist. By implementing the recommended **rate limiting** and **data filtering**, the organization can significantly improve its defense against automated attacks. These simple fixes will ensure the API is **hardened** and **protected** for all users.